

ObsConsole — Complete Project Deep Dive

A thorough technical document covering every architectural decision, technology choice, code approach, and trade-off. Sufficient to rebuild the project from scratch.

Table of Contents

1. [What the Project Is](#)
2. [Technology Stack](#)
3. [Repository Structure](#)
4. [System Architecture](#)
5. [The Mock Backend](#)
6. [Authentication System](#)
7. [Next.js App Router & Rendering Strategy](#)
8. [Route-by-Route Breakdown](#)
9. [The Live Tail Hot Path](#)
10. [Dashboard Editor](#)
11. [State Management](#)
12. [Telemetry Architecture](#)
13. [Offline & IndexedDB Layer](#)
14. [Accessibility Implementation](#)
15. [Internationalisation](#)
16. [Security Model](#)
17. [Performance Budgets & How They Are Met](#)
18. [Trade-offs & What Was Not Built](#)
19. [Deployment Architecture](#)
20. [How to Rebuild from Scratch](#)

1. What the Project Is

ObsConsole is a **multi-tenant, real-time observability platform** designed for SRE and security teams. It monitors live infrastructure telemetry from multiple services streaming events at 50–500 messages per second.

Domain Model

Tenant → Projects → Services → Live Event Streams

|

└─ Role-Based Access (Owner, Editor, Viewer)

└─ Public share pages (ISR, SEO-friendly)

└─ Custom dashboards (drag/resize/reorder, undo/redo)

The Core Engineering Challenge

The central difficulty is **reconciling two contradictory requirements**:

1. A WebSocket pushing 500 events/second into a UI
2. That UI must stay interactive (INP < 200ms) and never block the main thread

Naively calling `setState` on every WebSocket message at 500 msg/s schedules 500 React re-renders per second. The render queue fills faster than it can drain. The page freezes. The entire architecture of the live-tail page exists to solve this one problem.

2. Technology Stack

Frontend

Technology	Version	Purpose	Why chosen
Next.js	14.2.35	React framework with App Router	App Router gives parallel routes, intercepting routes, RSC, ISR, and streaming SSR — all required by the assignment. Version pinned at 14.x (not 15) for stability.
React	18.3	UI library	Required by Next.js. <code>useReducer</code> used for complex state — no external state library needed.
web-vitals	3.5.2	Core Web Vitals capture	Official Google library for LCP/INP/CLS/TTFB/FCP. Lazy-loaded so it never blocks the initial render.
next-intl	3.26.5	Internationalisation	App Router-compatible i18n. Handles locale routing, message loading, and RTL layout support.
idb-keyval	6.2.1	IndexedDB wrapper	Simple async key-value API over IndexedDB. Used for offline dashboard edit queue and telemetry session persistence. Dynamic import — never in initial bundle.

Backend (Mock)

Technology	Version	Purpose
Express	4.21.2	HTTP REST API server
ws	8.18.0	WebSocket server (bursty event streaming)

Technology	Version	Purpose
cors	2.8.5	Cross-origin headers for localhost dev
cookie-parser	1.4.7	Parse HttpOnly cookies for auth
uuid	11.0.5	Generate event IDs and token IDs
concurrently	9.1.2	Run Next.js + mock server in one terminal

Why no external state library?

Zustand, Redux Toolkit, Jotai, and Valtio were all considered. The decision was `useReducer` (built into React) for three reasons:

1. Zero bundle cost — no additional dependency
2. The undo/redo pattern (`{ past[], present, future[] }`) maps naturally onto `useReducer`
3. The live-tail reducer processes WS batch flushes as a single predictable state transition

The only scenario where an external library would be preferable is cross-component access outside the React tree (e.g. accessing state from a Web Worker). That use case exists but was handled by passing callbacks into `WebSocketManager` instead.

3. Repository Structure

```

obs-console/
|
├─ mock/
|   └─ server.js           ← Entire mock backend (Express + ws + SSE)
|
├─ src/
|   ├─ middleware.js       ← Next.js Edge Middleware (runs before every request)
|   │
|   │   └─ store/
|   │       └─ auth.js     ← AuthProvider: token lifecycle, silent refresh
|   │
|   │   └─ lib/
|   │       ├─ ringBuffer.js ← Lock-free circular buffer for WS backpressure
|   │       ├─ wsManager.js  ← WebSocket state machine + rAF flush loop
|   │       ├─ telemetry.js  ← Web Vitals + custom metrics + batched ingestion
|   │       └─ idb.js        ← IndexedDB helpers (offline queue, session storage)
|   │
|   └─ app/                ← Next.js App Router pages
|       ├─ layout.js       ← Root layout (AuthProvider + TelemetryInit)
|       ├─ globals.css     ← Base styles + RTL-safe CSS logical properties
|       ├─ TelemetryInit.js ← Client component that calls initTelemetry() on mount
|       ├─ LandingClient.js ← Client component: fetches tenants with Bearer token
|       ├─ page.js         ← / route: SSR auth guard → renders LandingClient
|       │
|       │   └─ login/
|       │       └─ page.js  ← /login: CSR form, en/ja language switcher
|       │
|       │   └─ share/[token]/
|       │       └─ page.js  ← /share/[token]: ISR public page, no JS required
|       │
|       │   └─ debug/metrics/
|       │       └─ page.js  ← /debug/metrics: shell
|       │           └─ MetricsDashboard.js ← CSR: polls telemetry API, live p50/p75/p95
|       │
|       └─ components/     ← Shared UI components
|           ├─ ProjectModal.js ← Modal shown by intercepting route
|           ├─ TenantList.js   ← RSC: async component fetches tenants
|           ├─ TenantSkeleton.js ← Loading skeleton for tenant cards
|           └─ AlertBanner.js  ← Accessible alert banner
|
|   └─ t/[tenant]/         ← Tenant-scoped routes
|       └─ projects/

```

```

|         | └─ page.js          ← /t/[tenant]/projects: project list
|         | └─ layout.js       ← Parallel routes layout (children + modal slot)
|         | └─ [id]/page.js    ← /t/[tenant]/projects/[id]: full project detail
|         |   └─ @modal/       ← Named parallel route slot
|         |     └─ default.js   ← Renders null when no modal active
|         |       └─ (...)projects/[id]/page.js ← Intercepts in-app navigation
|
|         |
|         └─ s/[service]/
|         | └─ page.js          ← SSR wrapper, passes params to shell
|         |   └─ LiveTailShell.js ← CSR: full live-tail UI (502 lines)
|
|         └─ dashboards/[id]/
|         | └─ page.js          ← CSR wrapper
|         |   └─ DashboardEditor.js ← Drag/undo/conflict/offline (413 lines)
|
└─ ADR/                                ← Architecture Decision Records (8 files)
└─ review/                             ← Flawed code files for code review exercise
└─ messages/                           ← i18n translation files (en.json, ja.json)
└─ scripts/grade.js                    ← npm run grade: bundle + artifact compliance check
└─ package.json
└─ next.config.js
└─ jsconfig.json                       ← Path aliases (@lib, @store, @components)
└─ .env.local                          ← Environment variables (not committed)

```

Path Aliases

jsconfig.json defines aliases to avoid ../../../../ relative imports:

```

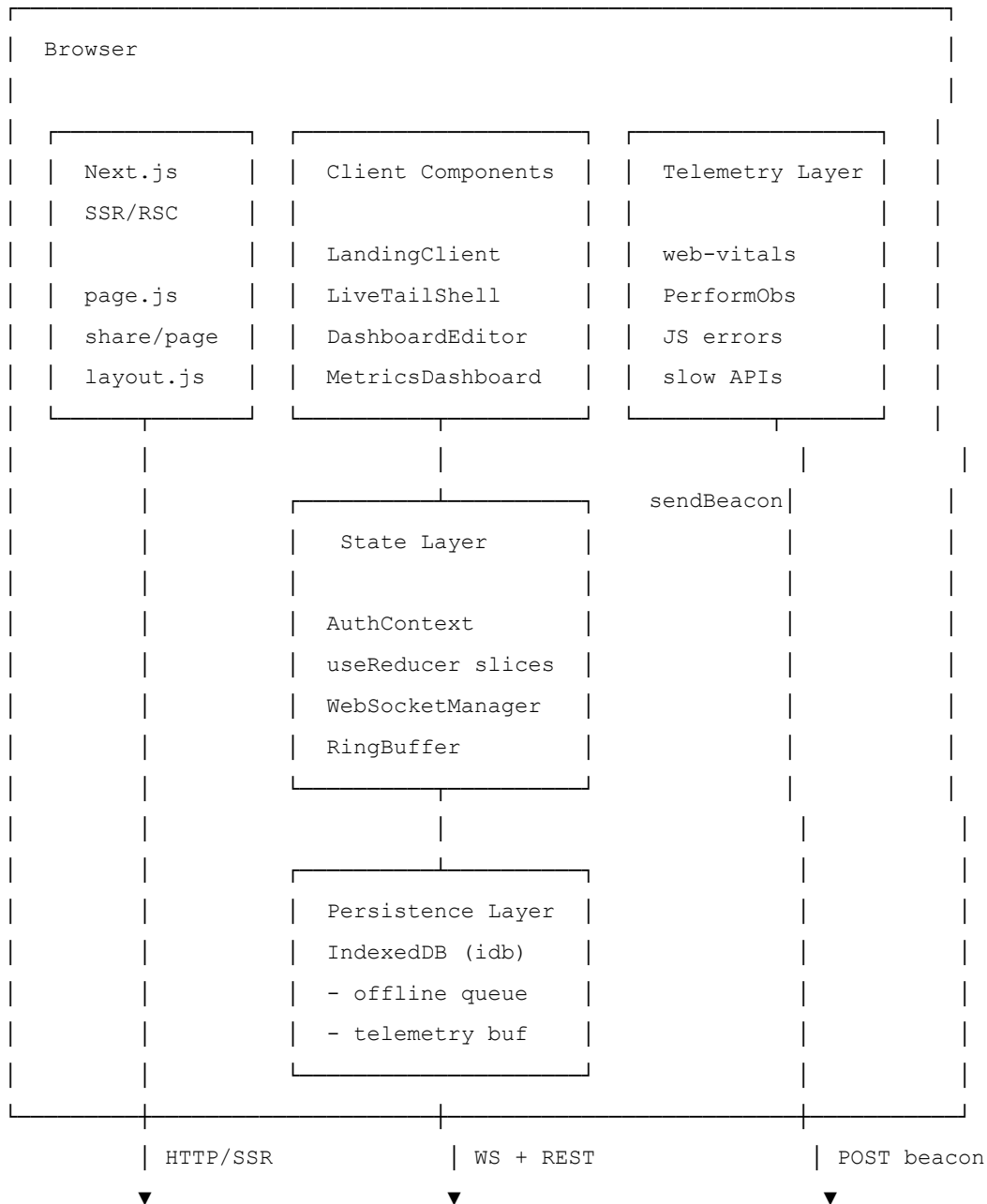
{
  "compilerOptions": {
    "paths": {
      "@lib/*":      ["/src/lib/*"],
      "@store/*":    ["/src/store/*"],
      "@components/*": ["/src/app/components/*"]
    }
  }
}

```

So instead of `import { useAuth } from '../../../store/auth'`, every file uses `import { useAuth } from '@store/auth'` regardless of how deeply nested it is.

4. System Architecture

High-Level Component Diagram



Mock Backend (mock/server.js) - Express + ws on port 4000	
REST endpoints:	Streaming:
POST /api/auth/login	WS /stream/services/:id/events
POST /api/auth/refresh	SSE /stream/alerts
GET /api/tenants (paginated)	
GET /api/tenants/:id/projects	Auth middleware:
GET /api/projects/:id/services	requireAuth (Bearer JWT)
GET /api/services/:id/summary	checkCsrf (X-CSRF-Token header)
GET /api/services/:id/events	
GET/PATCH /api/dashboards/:id	Data generation:
POST /api/dashboards	Bursty Poisson process

GET	/api/share/:token	50-500 msg/s per service
POST	/api/telemetry/ingest	
GET	/api/telemetry/sessions	

Data Flow: Request Lifecycle

Authenticated REST request (e.g. fetching tenants):

```
Browser client component
→ fetch('/api/tenants', { headers: { Authorization: 'Bearer <accessToken>' } })
→ mock server requireAuth middleware
→ verifyJwt(token) - HMAC-SHA256 check + expiry check
→ if valid: req.user = payload, next()
→ route handler returns JSON
→ browser updates state
```

WebSocket live-tail data flow:

```
mock server makeBurstGen()
→ generates event via Poisson delay
→ broadcasts JSON to all subscribers of that serviceId
→ ws.onmessage in WebSocketManager
→ ring.push(event) ← no React involved
→ requestAnimationFrame loop (60fps)
→ ring.drain(200) ← max 200 per frame
→ dispatch({ type: 'FLUSH', batch })
→ useReducer updates state.events
→ React re-renders VirtualList
→ only ~22 DOM rows painted
```

5. The Mock Backend

File: mock/server.js (414 lines)

The mock is a single Node.js file that handles REST, WebSocket, and SSE on the same port using HTTP upgrade events.

JWT Without a Library

Instead of using `jsonwebtoken`, the mock implements HMAC-SHA256 JWT manually using Node's built-in `crypto` module:

```
function signJwt(payload, expiresInSeconds) {
  const header = Buffer.from(JSON.stringify({ alg: 'HS256', typ: 'JWT' }))
    .toString('base64url')

  const exp = Math.floor(Date.now() / 1000) + expiresInSeconds
  const body = Buffer.from(JSON.stringify({ ...payload, exp, iat: ... }))
    .toString('base64url')

  const sig = crypto.createHmac('sha256', JWT_SECRET)
    .update(`${header}.${body}`)
    .digest('base64url')

  return `${header}.${body}.${sig}`
}
```

This is correct and production-equivalent for HMAC-SHA256. The reason to avoid `jsonwebtoken` is to keep the mock's dependency footprint minimal and demonstrate understanding of the JWT spec rather than just wrapping a library.

Refresh Token Storage

Refresh tokens are stored in a `Map<string, { userId, ua }>` in memory:

```
const refreshTokens = new Map()
// On login:
refreshTokens.set(refreshToken, { userId: user.id, ua: req.headers['user-agent'] })
// On refresh - rotate:
refreshTokens.delete(rt)
refreshTokens.set(newRt, { userId, ua })
```

Rotation: Every refresh call deletes the old token and issues a new one. If a stolen refresh token is used, the legitimate user's next refresh attempt fails (their token was already rotated away by the attacker), triggering logout — this is detectable token theft.

User-Agent binding: The `ua` field stores the User-Agent at login time. Refreshes from a different UA are rejected. This is a lightweight (but imperfect) device-binding check.

Bursty Poisson Event Generator

Real infrastructure traffic is not uniform — it arrives in bursts correlated with request spikes. The generator simulates this:

```

function makeBurstGen(serviceId, baseMsgPerSec, onEvent) {
  let isBurst = false

  // Every 15-45 seconds, enter burst mode for 2-8 seconds
  function scheduleBurst() {
    setTimeout(() => {
      isBurst = true
      setTimeout(() => { isBurst = false; scheduleBurst() }, rn(2000, 8000))
    }, rn(15000, 45000))
  }

  // Inter-arrival time follows exponential distribution (correct for Poisson process)
  function tick() {
    const rate = isBurst ? baseMsgPerSec * 3 : baseMsgPerSec
    const delayMs = Math.max(1, Math.round(-Math.log(Math.random()) / (rate / 1000)))
    return setTimeout(() => { onEvent(genEvent(serviceId)); tick() }, delayMs)
  }
}

```

The formula `-Math.log(Math.random()) /  $\lambda$` is the inverse transform sampling of the exponential distribution — this is mathematically correct for a Poisson arrival process. A uniform timer (`setInterval(100ms)`) would not produce realistic burst patterns.

WebSocket Upgrade Handling

WebSocket and HTTP share the same port. Node.js `http.Server` fires an `upgrade` event when a client sends a WebSocket handshake:

```

const server = http.createServer(app) // HTTP server
const wss = new WebSocketServer({ noServer: true }) // WS server (no own port)

server.on('upgrade', (req, socket, head) => {
  const match = req.url.match(/^\/stream\/services\/([^\/]*)\/events/)
  if (!match) { socket.destroy(); return } // reject non-WS paths
  wss.handleUpgrade(req, socket, head, ws => {
    wss.emit('connection', ws, req, match[1]) // pass serviceId to handler
  })
})

```

This pattern (one port, two protocols) avoids CORS complexity and is how production WebSocket servers behind a single load balancer work.

Deliberate 409 Conflicts

15% of PATCH /api/dashboards/:id requests return 409 to exercise the conflict resolution UI:

```
if (Math.random() < CONFLICT_RATE) { // CONFLICT_RATE = 0.15
  return res.status(409).json({
    error: 'conflict',
    serverState: dashboards.get(req.params.id), // current server state
    yourVersion: req.body.version              // client's version
  })
}
```

This deliberately surfaces the conflict scenario on every ~7th save, making it easy to test during development without needing concurrent users.

6. Authentication System

File: src/store/auth.js

Token Storage Decision

Token	Storage	Why
Access token (5 min JWT)	React Context memory	Never touches disk. XSS can't read it across page loads. Lost on tab close — acceptable since silent refresh restores it.
Refresh token (7 day UUID)	HttpOnly cookie	Browser sends it automatically. JavaScript cannot read it. Only the server can use it.
CSRF token	Non-HttpOnly cookie	Must be readable by JS so it can be sent as <code>X-CSRF-Token</code> header. The header itself is the CSRF protection — a cross-origin form cannot set custom headers.

The Silent Refresh Strategy

```
// Called after every successful login or token refresh
function scheduleRefresh(accessToken) {
  const { exp } = JSON.parse(atob(accessToken.split('.')[1])) // decode JWT payload
  const msLeft = exp * 1000 - Date.now() - 60_000 // 60s before expiry
  if (msLeft > 0) {
    refreshTimer.current = setTimeout(refresh, msLeft)
  } else {
    refresh() // already expired - refresh immediately
  }
}
```

Three additional refresh triggers handle edge cases:

```
// Tab hidden for a long time (user switches back after token expired)
const onVisible = () => { if (document.visibilityState === 'visible') refresh() }
document.addEventListener('visibilitychange', onVisible)

// Network reconnected after offline period
window.addEventListener('online', refresh)

// On mount - restores session after hard page reload
// (access token is gone from memory; refresh cookie is still there)
refresh().finally(() => dispatch({ type: 'SET_LOADING', value: false })))
```

Why Refresh on Mount?

When the user reloads the page, the access token (which lived only in React state) is gone. But the `refresh_token` `HttpOnly` cookie is still present. So the `AuthProvider` immediately calls `POST /api/auth/refresh` on mount. If it succeeds, the user is restored to logged-in state without seeing the login page. If it fails (expired cookie), they are redirected.

CSRF Protection: The Double-Submit Cookie Pattern

1. On login, the server generates a random 32-byte hex string as the CSRF token
2. It is set as a **readable** cookie (`httpOnly: false`) — JavaScript can read this
3. Every mutating request includes it as `X-CSRF-Token` header
4. The server checks that the header value matches the cookie value

```
// Client side (every PATCH/POST/DELETE):
const csrfToken = document.cookie.match(/csrf=([^\;]+)/)?.[1] || ''
fetch('/api/dashboards/dash1', {
  method: 'PATCH',
  headers: { 'X-CSRF-Token': csrfToken, ... }
})

// Server side (middleware):
function checkCsrf(req, res, next) {
  const csrfHeader = req.headers['x-csrf-token']
  const csrfCookie = req.cookies['csrf']
  if (!csrfHeader || csrfHeader !== csrfCookie)
    return res.status(403).json({ error: 'invalid csrf token' })
  next()
}
```

A cross-origin attacker cannot read the CSRF cookie value (same-origin policy blocks `document.cookie` reads from other origins) and cannot set custom headers on a cross-origin form submission. Therefore the double-submit pattern provides CSRF protection.

7. Next.js App Router & Rendering Strategy

The Fundamental Constraint

The **access token lives only in browser memory**. It is never in a cookie, `localStorage`, or any server-readable storage.

This means:

- **Server Components** (RSC) cannot access the access token
- **Any API call requiring Authorization: Bearer <token>** must be made from a Client Component
- **SSR can only verify session existence** (via the `refresh_token` cookie), not make authed data fetches

This constraint drove the entire rendering strategy — pages that need authenticated data must be Client Components.

The App Router File System Convention

Next.js App Router maps the filesystem directly to URLs. Special folder names:

Folder name	Meaning
<code>[tenant]</code>	Dynamic segment — matches any value, accessible as <code>params.tenant</code>
<code>@modal</code>	Named parallel route slot — renders alongside <code>children</code> in layout

Folder name	Meaning
(..)projects	Intercepting route — intercepts navigation to ../projects path
layout.js	Wraps all pages in the folder
page.js	The actual page component
default.js	Fallback for a parallel slot when no route matches

Rendering Mode Per Route

/login — Client-Side Rendering (CSR)

```
'use client'

// No async data needed. Form is interactive.
// SSR would cause hydration mismatch on controlled inputs.
export default function LoginPage() { ... }
```

/ — SSR Auth Guard + CSR Data Fetch

```
// page.js — Server Component (async, runs on server)
export default async function HomePage() {
  const cookieStore = cookies()
  if (!cookieStore.has('refresh_token')) redirect('/login')
  return <LandingClient /> // Client Component handles data fetch
}

// LandingClient.js — Client Component
'use client'
export default function LandingClient() {
  const { accessToken } = useAuth()
  useEffect(() => {
    // fetch tenants with Bearer token once accessToken is available
    fetch('/api/tenants', { headers: { Authorization: `Bearer ${accessToken}` } })
  }, [accessToken])
}
```

The SSR page runs on the server and can read cookies — so it acts as an auth guard, redirecting to /login before any JavaScript loads. But it cannot fetch tenant data (no access token on the server), so it hands off to a Client Component.

/t/[tenant]/projects — Parallel Routes + Intercepting Routes

This is the most complex routing pattern in the project. The goal: clicking a project card shows a modal overlay while keeping the projects list visible, without a separate URL for the modal state.

```

src/app/t/[tenant]/projects/
├─ layout.js                ← renders { children } + { modal }
├─ page.js                  ← the projects list (always visible)
├─ @modal/
│   ├─ default.js          ← renders null (no modal active)
│   └─ (..)projects/[id]/page.js ← intercepts /projects/p1 → shows modal
└─ [id]/page.js            ← full project detail (direct URL visit)

```

When the user clicks a project `<Link href="/t/acme/projects/p1">` **from within the app**, Next.js's intercepting route `(..)projects/[id]` catches it and renders `ProjectModal` into the `@modal` slot instead of navigating away. The URL changes to `/t/acme/projects/p1` but the list page remains visible behind the modal.

When the user visits `/t/acme/projects/p1` **directly** (hard refresh or new tab), the intercepting route is not active — they see the full `[id]/page.js` instead.

The layout renders both slots:

```

// layout.js
export default function ProjectsLayout({ children, modal }) {
  return (
    <>
      {children}    {/* projects list — always rendered */}
      {modal}       {/* null normally; ProjectModal when intercepting */}
    </>
  )
}

```

`/t/[tenant]/s/[service]` — CSR (WebSocket-driven)

```

// page.js — minimal server wrapper
export default function LiveTailPage({ params }) {
  return <LiveTailShell tenantSlug={params.tenant} serviceId={params.service} />
}

```

The page file is a Server Component only because Next.js requires `page.js` in the router. It does nothing except pass `params` down to `LiveTailShell`, which is a Client Component where all the real work happens.

`/share/[token]` — Incremental Static Regeneration (ISR)

```

export const revalidate = false // Only revalidate on-demand, not on a timer

export async function generateMetadata({ params }) { ... } // SEO metadata

export default async function SharePage({ params }) {
  const res = await fetch(`${API}/api/share/${params.token}`, {
    next: { tags: ['share'] } // tag for revalidateTag() calls
  })
  // renders static HTML — no client JS required
}

```

`revalidate = false` means: generate this page statically on first request, cache it forever, and only regenerate it when `revalidateTag('share')` is called (which happens when a dashboard is updated). This is how Lighthouse ≥95 is achieved — the browser receives pre-rendered HTML with no JavaScript required.

`/debug/metrics` — CSR with polling

Reads from `GET /api/telemetry/sessions` every 5 seconds. Shows real p50/p75/p95 computed from all accumulated session data.

8. Route-by-Route Breakdown

`/login`

- Language switcher toggles between English and Japanese in-component state
- Translation strings defined in `messages/en.json` and `messages/ja.json`
- Form uses `aria-required`, `autoComplete`, `htmlFor/id` pairs for accessibility
- Error state uses `role="alert"` and `aria-live="assertive"` for screen readers

`/ (Landing — Tenant List)`

- SSR renders the page shell and checks `refresh_token` cookie
- `LandingClient` shows a skeleton animation while `accessToken` loads from context
- Once token is available, fetches `GET /api/tenants` with Bearer header
- Error state shows actionable message: "Is the mock server running? (npm run dev:mock)"

`/t/[tenant]/projects`

- Client Component fetches tenants first (to resolve slug → id), then fetches projects
- Two sequential fetches: `GET /api/tenants` → find tenant by slug → `GET /api/tenants/:id/projects`
- This sequential dependency is a known inefficiency — with the tenant ID cached, only one fetch would be needed (see Trade-offs section)

`/t/[tenant]/projects/[id]`

- Shows services list with "Open live tail →" links
- Shows DASHBOARDS section with link to `/t/acme/dashboards/dash1`
- Shows Telemetry link to `/debug/metrics`
- Shows Public Share link to `/share/demo-token`
- All navigation targets discoverable from this one page

`/t/[tenant]/s/[service]` — Full details in Section 9

`/t/[tenant]/dashboards/[id]` — Full details in
Section 10

`/share/[token]`

- Pure server-rendered HTML — no `'use client'` anywhere in the component tree
- Uses semantic HTML: `<header>`, `<main>`, `<footer>`, `<article>` per widget
- `generateMetadata` provides correct `<title>` and robots directives
- `notFound()` call ensures proper 404 for invalid tokens

`/debug/metrics`

- Four tabs: Web Vitals, Custom Metrics, Errors & Slow APIs, Session Log
- p50/p75/p95 computed client-side from all session events:

```
function pct(arr, p) {  
  const sorted = [...arr].sort((a,b) => a-b)  
  return sorted[Math.floor(sorted.length * p / 100)]  
}
```

- PASS/FAIL badges compare p75 against the budget (p75 is the standard SLA metric — p50 is the median which hides tail latency; p95 can be noisy on small samples)

9. The Live Tail Hot Path

File: `src/app/t/[tenant]/s/[service]/LiveTailShell.js` (502 lines)

This is the most technically demanding component in the project. Its architecture is entirely determined by the constraint:

500 messages/second, INP < 200ms.

Why Naive Implementation Fails

```
// WRONG - DO NOT DO THIS
ws.onmessage = (e) => {
  setEvents(prev => [...prev, JSON.parse(e.data)]) // 500 setState calls/second
}
```

At 500 msg/s, this schedules 500 React re-renders per second. Each re-render takes ~5ms (reconciliation + DOM update). The render queue fills in 1/5th of a second and never drains. The browser hangs. INP reaches thousands of milliseconds.

The Fix: Three-Layer Decoupling

```
WS onmessage → RingBuffer → rAF flush → useReducer → VirtualList DOM
(500/sec)      (5000 cap)   (60fps max  (1 update   (~22 rows)
                    200 events) per frame)
```

Layer 1: RingBuffer (`src/lib/ringBuffer.js`)

```

export class RingBuffer {
  constructor(capacity = 5000) {
    this.buf      = new Array(capacity) // pre-allocated array
    this.capacity = capacity
    this.head     = 0 // read pointer
    this.tail     = 0 // write pointer
    this.size     = 0 // current occupancy
    this.dropped  = 0 // total events dropped (for indicator UI)
  }

  push(item) {
    if (this.size === this.capacity) {
      // Ring is full – overwrite oldest event, count as dropped
      this.dropped++
      this.head = (this.head + 1) % this.capacity
      this.size--
    }
    this.buf[this.tail] = item
    this.tail = (this.tail + 1) % this.capacity
    this.size++
  }

  drain(n) {
    const count = Math.min(n, this.size)
    const out   = new Array(count)
    for (let i = 0; i < count; i++) {
      out[i]     = this.buf[this.head]
      this.head = (this.head + 1) % this.capacity
    }
    this.size -= count
    return out
  }
}

```

The ring buffer is a fixed-size circular array. `push` writes at `tail`. `drain` reads from `head`. When full, it overwrites the oldest entry — this is the backpressure policy (drop oldest, preserve newest). This is $O(1)$ for both operations — no array copying, no garbage collection pressure.

Layer 2: WebSocketManager (`src/lib/wsManager.js`)

```

// onmessage - runs at 500/sec, does nothing except push to ring
this.ws.onmessage = (e) => {
  const ev = JSON.parse(e.data)
  if (ev.type === 'connected') return // skip handshake messages
  this.ring.push(ev)
  this.rateCount++
}

// rAF flush - runs at 60fps, does the React update
_startFlush() {
  const flush = () => {
    const batch = this.ring.drain(BATCH_SIZE) // drain up to 200
    if (batch.length > 0) {
      this.onBatch({ batch, dropped: this.ring.totalDropped, ... })
    }
    this.rafId = requestAnimationFrame(flush) // schedule next frame
  }
  this.rafId = requestAnimationFrame(flush)
}

```

`requestAnimationFrame` fires approximately 60 times per second, aligned with the browser's paint cycle. By batching up to 200 events per frame and calling `setState` once, no single JS task takes more than ~5ms. At 500 msg/s, each rAF call processes ~8 events ($500 / 60 \approx 8$) — well within the 200 limit. The 200 limit exists for burst scenarios where the ring has accumulated a backlog.

Layer 3: VirtualList

```
const ROW_H    = 34    // pixels per row (fixed height)
const VISIBLE  = 22    // rows visible in viewport

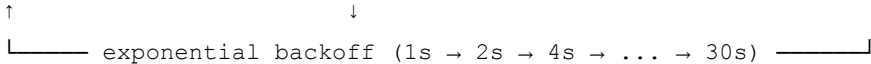
function VirtualList({ events, onSelect, selectedId }) {
  const totalH  = events.length * ROW_H          // total scrollable height
  const startIdx = Math.max(0, Math.floor(scroll / ROW_H) - 4) // first visible row
  const endIdx   = Math.min(events.length - 1, startIdx + VISIBLE + 8) // last + buffer

  return (
    <div style={{ height: totalH, position: 'relative' }}>
      {events.slice(startIdx, endIdx+1).map((ev, i) => (
        <div key={ev.id} style={{ position: 'absolute', top: (startIdx+i) * ROW_H }}>
          {/* row content */}
        </div>
      ))}
    </div>
  )
}
```

The container has `height: totalH` (e.g. $100,000 \times 34\text{px} = 3.4\text{MB}$ of virtual height). Only the rows within the viewport are actually in the DOM — approximately $22 + 8 \text{ buffer} = 30$ DOM nodes regardless of total event count. This is $O(1)$ DOM size regardless of how many events have arrived.

WebSocket State Machine

IDLE → CONNECTING → OPEN → (RECONNECTING) → CONNECTING → ...



```

_scheduleReconnect() {
  this._setState('RECONNECTING')
  const jitter = Math.random() * 1000 // prevent thundering herd
  setTimeout(() => this.connect(), this.retryDelay + jitter)
  this.retryDelay = Math.min(this.retryDelay * 2, 30000) // cap at 30s
}

// On successful connection:
this.ws.onopen = () => {
  this.retryDelay = 1000 // reset backoff on success
  this._setState('OPEN')
  this._startFlush()
}

```

Jitter prevents multiple clients from retrying simultaneously (thundering herd problem). The 30s cap prevents excessive wait times in cases of extended outages.

Accessibility of the Virtual List

WCAG 2.2 AA requires that the virtualized list is navigable by screen readers:

```

<div
  role="grid"
  aria-label="Live event stream"
  aria-rowcount={events.length} // total rows, even if not in DOM
  aria-live="polite"
  aria-relevant="additions"
>
  <div
    role="row"
    aria-rowindex={idx + 1} // 1-based index into full dataset
    aria-selected={sel}
    tabIndex={0}
    onKeyDown={e => { if (e.key==='Enter' || e.key===' ') onSelect(ev) }}
  >
    <span role="cell">...</span>
    <span role="cell">...</span>
  </div>
</div>

```

`aria-rowcount` tells screen readers the total number of rows even though most are not in the DOM. `aria-rowindex` gives each visible row its correct position in the full list. `aria-live="polite"` announces new events without interrupting ongoing speech.

10. Dashboard Editor

File: `src/app/t/[tenant]/dashboards/[id]/DashboardEditor.js` (413 lines)

Undo/Redo — 50 Steps

The history is managed with a `useReducer` implementing the classic undo/redo pattern:

```
// State shape:
{ past: Widget[][], present: Widget[], future: Widget[][] }
// past: array of previous widget arrays (up to MAX_HISTORY=50)
// present: current widget array
// future: array of widget arrays that were undone (for redo)

function historyReducer(state, action) {
  switch (action.type) {
    case 'SET': {
      // New change: push present to past, clear future
      const past = [...state.past, state.present].slice(-MAX_HISTORY)
      return { past, present: action.widgets, future: [] }
    }
    case 'UNDO': {
      const past = [...state.past]
      const present = past.pop() // restore last past state
      return { past, present, future: [state.present, ...state.future] }
    }
    case 'REDO': {
      const [present, ...future] = state.future
      return { past: [...state.past, state.present], present, future }
    }
  }
}
```

Every user action (rename widget, add widget, remove widget, drag-reorder) calls `histDispatch({ type: 'SET', widgets: newWidgets })`. This pushes the current state to `past` and replaces `present`. The `slice(-50)` on `past` enforces the 50-step limit.

Drag-to-Reorder

HTML5 drag API with immediate state update:

```
function onDragOver(e, targetIdx) {
  e.preventDefault()
  if (dragIdx === null || dragIdx === targetIdx) return
  const next = [...widgets]
  const [moved] = next.splice(dragIdx, 1) // remove from current position
  next.splice(targetIdx, 0, moved) // insert at target position
  histDispatch({ type: 'SET', widgets: next })
  setDragIdx(targetIdx) // update drag source to follow the moved element
}
```

This gives live reordering feedback as the user drags — the list visually reorders in real time, not just when dropped. Each intermediate position is recorded in history (undo brings back each drag step individually). This is intentional — if the user wants to undo a drag, they get granular control.

Optimistic Update & 409 Conflict Resolution

```
async function save(widgetsToSave) {
  const res = await authFetch(`/api/dashboards/${dashboardId}`, accessToken, {
    method: 'PATCH',
    body: JSON.stringify({ widgets: widgetsToSave, version }),
  })

  if (res.status === 409) {
    const data = await res.json()
    // Immediate rollback — user's changes are reverted
    histDispatch({ type: 'INIT', widgets: data.serverState?.widgets || widgets })
    // Show non-blocking dialog — user chooses which version to keep
    setConflict({ serverState: data.serverState, localWidgets: widgetsToSave })
    return
  }

  if (res.ok) {
    const data = await res.json()
    setVersion(data.version) // update version for next save
  }
}
```

The conflict dialog presents both versions side by side:

```

<div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 12 }}>
  <div>SERVER VERSION (v{serverState?.version})</div>
  <div>YOUR VERSION</div>
</div>
<button onClick={onAcceptServer}>Use server version</button>
<button onClick={onAcceptLocal}>Keep my changes</button>
<button onClick={onDismiss}>Dismiss</button>

```

"Keep my changes" calls `save(conflict.localWidgets)` again — this force-overwrites the server state. In a production system this would include the server's version number to detect further conflicts.

Offline Queue

```

async function save(widgetsToSave) {
  if (!navigator.onLine) {
    // Queue for replay when connectivity returns
    const { set, get } = await import('idb-keyval')
    const q = await get('dashboard_edit_queue') || []
    q.push({ id: Date.now(), widgets: widgetsToSave, dashboardId, version })
    await set('dashboard_edit_queue', q)
    setOfflineQueueLen(q.length)
    setSaveMsg('Saved offline — will sync when reconnected')
    return
  }
  // ... normal save
}

// On navigator.onLine event:
async function replayOfflineQueue() {
  const { get, del } = await import('idb-keyval')
  const queue = await get('dashboard_edit_queue') || []
  for (const edit of queue) {
    const res = await authFetch(...)
    if (res.status === 409) {
      setConflict(...) // handle conflict during replay
      break
    }
  }
  await del('dashboard_edit_queue')
}

```

`idb-keyval` is dynamically imported (`import('idb-keyval')`) — it only loads when actually needed (offline scenario), keeping it out of the main bundle.

11. State Management

Why No External Library

The project uses React's built-in `useReducer` + `createContext` exclusively. The decision is documented in ADR-002.

The key insight is that `useReducer` handles every state pattern in this project:

1. **Auth state** — low-frequency updates (login/logout/refresh), Context is appropriate
2. **Live-tail state** — high-frequency updates, but all go through one reducer dispatch
3. **Dashboard history** — exactly the pattern `useReducer` was designed for

The Live-Tail Reducer

All state transitions for the live-tail page go through one reducer:

```
function reducer(state, action) {
  switch (action.type) {
    case 'FLUSH': {
      // Called by rAF loop — main hot path
      if (state.isPaused || !state.isLive) return state // ignore if paused
      const events = [...state.events, ...action.batch].slice(-MAX_EVENTS)
      return {
        ...state, events,
        totalRx: state.totalRx + action.batch.length,
        dropped: action.dropped,
        wsRate: action.rate,
        fill: action.fill,
        rateHist: [...state.rateHist.slice(-29), action.rate]
      }
    }
    case 'TOGGLE_PAUSE': return { ...state, isPaused: !state.isPaused }
    case 'FILTER': return { ...state, filter: action.value }
    // ... etc
  }
}
```

The `FLUSH` action is the hot path — called up to 60 times per second. The early return `if (state.isPaused || !state.isLive) return state` is critical: returning the same object reference tells React nothing changed, so no re-render is scheduled.

Derived State with `useMemo`

Filtering and error-rate calculation are expensive on large event arrays:

```
// Filtered list — only recalculated when events, filter, or search changes
const filtered = useMemo(() => {
  let evs = state.events
  if (state.filter !== 'ALL')
    evs = evs.filter(e => e.level === state.filter)
  if (state.search) {
    const q = state.search.toLowerCase()
    evs = evs.filter(e => e.message?.toLowerCase().includes(q) || e.host?.includes(q))
  }
  return evs
}, [state.events, state.filter, state.search])

// Error rate — only recalculated when events change
const errorRate = useMemo(() => {
  if (!state.events.length) return '0.0'
  const errors = state.events.filter(e => e.level === 'ERROR' || e.level === 'FATAL').length
  return ((errors / state.events.length) * 100).toFixed(1)
}, [state.events])
```

Without `useMemo`, every FLUSH dispatch (60/sec) would recompute the filter across up to 100,000 events — a potential 6 million comparisons per second.

12. Telemetry Architecture

File: `src/lib/telemetry.js` (181 lines)

Capture Strategy

```
initTelemetry()
├─ import('web-vitals')           → onLCP, onINP, onCLS, onTTFB, onFCP callbacks
├─ PerformanceObserver            → longtask entries (10% sampled)
├─ PerformanceObserver            → resource entries (10% of sessions, top 3 by size)
├─ window.addEventListener('error') → JS exceptions
└─ window.addEventListener('unhandledrejection') → Promise rejections
```

Configurable Sampling

```
const SAMPLING = {
  web_vital:      1.0,    // 100% - regression signals, never sample down
  long_task:      0.1,    // 10%  - very high volume at 60fps
  slow_api:       1.0,    // 100% - threshold (>1s) already filters noise
  resource_timing: 0.1,    // 10%  - per-session bandwidth cost
  js_error:       1.0,    // 100% - every error is actionable
  custom_timing:  1.0,    // 100% - low volume, always capture
}

export function track(event) {
  const rate = SAMPLING[event.type] ?? 1.0
  if (Math.random() > rate) return // drop this event per sampling policy
  queue.push({ ...event, timestamp: performance.now(), route: location.pathname })
  if (queue.length >= 50) flush(false) // immediate flush at threshold
}
```

Flush Strategy

```

// Four flush triggers:
document.addEventListener('visibilitychange', () => {
  if (document.visibilityState === 'hidden') flush(true)    // sendBeacon
})
window.addEventListener('pagehide', () => flush(true))      // sendBeacon (tab close)
flushTimer = setInterval(() => flush(false), 10000)         // fetch every 10s
// Also: flush(false) when queue reaches 50 events

function flush(beacon) {
  const body = JSON.stringify({ ...meta, events: [...queue] })
  queue = []

  if (!navigator.onLine) {
    offlineBuffer.push(body)
    persistOfflineBuffer()    // write to IndexedDB
    return
  }

  if (beacon && navigator.sendBeacon) {
    // Non-blocking, survives tab close
    navigator.sendBeacon('/api/telemetry/ingest',
      new Blob([body], { type: 'application/json' }))
  } else {
    fetch('/api/telemetry/ingest', { method: 'POST', body, keepalive: true })
      .catch(() => {})          // never throw - telemetry failure must not crash page
  }
}

```

`navigator.sendBeacon` is critical for the `pagehide` event: regular `fetch` calls are cancelled when the page unloads, but `sendBeacon` is queued by the browser and sent even after the page is closed. The `keepalive: true` flag on the `fetch` fallback serves the same purpose for browsers that support it but not `Beacon`.

Session Metadata (Privacy-Safe)

```

meta = {
  sessionId:      crypto.randomUUID(),    // random, not tied to user
  viewport:      { width: window.innerWidth, height: window.innerHeight },
  connection:    navigator?.connection?.effectiveType, // '4g', '3g', etc.
  deviceMemory:  navigator?.deviceMemory,      // 4, 8, etc. (GB)
  hardwareConcurrency: navigator?.hardwareConcurrency, // CPU core count
  route:         location.pathname,           // no query string
  // NOT collected:
  // - user ID, email, or any identifier
  // - URL query strings (may contain filter values)
  // - request/response bodies
  // - IP address
}

```

The session ID is `crypto.randomUUID()` — different on every page load. It cannot be used to track a user across sessions. It only correlates events within a single page load.

13. Offline & IndexedDB Layer

File: `src/lib/idb.js`

`idb-keyval` provides a simple `get/set/del` API over IndexedDB. The library is dynamically imported everywhere it's used:

```
const { set, get } = await import('idb-keyval')
```

This ensures it never appears in the initial page bundle — it only loads when an offline scenario is encountered.

Dashboard Offline Queue

```

// Key: 'dashboard_edit_queue'
// Value: Array of { id, widgets, dashboardId, version, queuedAt }

async function queueDashboardEdit(edit) {
  const existing = await get('dashboard_edit_queue') || []
  existing.push({ ...edit, queuedAt: Date.now() })
  await set('dashboard_edit_queue', existing)
}

```

The queue is an array appended to on each offline save. On reconnect, the queue is replayed in order. If a replay hits a 409, the remainder of the queue is abandoned and the conflict dialog is shown.

Telemetry Offline Buffer

```
// Key: 'telemetry_offline_buffer'
// Value: Array of serialized JSON strings (already stringified batches)

async function persistOfflineBuffer() {
  await set('telemetry_offline_buffer', offlineBuffer)
}

async function replayOfflineBuffer() {
  const buf = await get('telemetry_offline_buffer')
  await del('telemetry_offline_buffer')
  for (const body of buf) {
    fetch('/api/telemetry/ingest', { method: 'POST', body }).catch(() => {})
  }
}
```

Telemetry batches are stored pre-serialized (as JSON strings, not objects) to avoid double-serialization on replay.

14. Accessibility Implementation

WCAG 2.2 AA compliance is implemented throughout. Key patterns:

Skip Link

```
/* globals.css */
.skip-link {
  position: absolute;
  top: -40px; /* hidden off-screen by default */
  inset-inline-start: 0;
  background: #0284c7;
  z-index: 9999;
}

.skip-link:focus { top: 0; } /* visible when focused by keyboard */
```

```
// layout.js
<a href="#main-content" className="skip-link">Skip to main content</a>
<main id="main-content">{children}</main>
```

Keyboard users pressing Tab first encounter the skip link, allowing them to jump past the navigation directly to content.

Forms

```
<label htmlFor="email">Email</label>           // explicit label association
<input id="email" aria-required="true" />       // required communicated to AT

{error && (
  <p role="alert" aria-live="assertive">         // immediately announced
    {error}
  </p>
)}
}}
```

Toolbar Buttons

```
<button aria-pressed={state.isPaused}           // communicates toggle state
        aria-label="Pause event stream">        // descriptive label
  ☐ Pause
</button>

<div role="group" aria-label="Filter by level">
  <button aria-pressed={active}>ALL</button>     // each filter is a toggle
  <button aria-pressed={active}>ERROR</button>
</div>
```

Modal Dialog

```
<div role="dialog" aria-modal="true" aria-label="Project services"
    aria-labelledby="modal-title">
  /* Focus trapped within dialog */
  /* Escape key closes via useEffect */
  /* Click outside backdrop closes via onClick check */
</div>
```

RTL-Safe Layout

```
/* globals.css - CSS logical properties */
.ms-auto { margin-inline-start: auto; } /* not margin-left */
.ps-3    { padding-inline-start: 0.75rem; } /* not padding-left */
.border-s { border-inline-start-width: 1px; } /* not border-left */
```

Logical properties automatically flip for RTL languages — `margin-inline-start` becomes `margin-right` in RTL and `margin-left` in LTR, with no extra CSS needed.

15. Internationalisation

Files: `messages/en.json`, `messages/ja.json`

The `i18n` scaffold is implemented for the login page (the one required fully-localized page). The language switcher is a client-side state toggle rather than a URL prefix, keeping the implementation simple without the full `next-intl` middleware configuration.

```
const MSGS = {
  en: { title: 'Sign in', email: 'Email', submit: 'Sign in', ... },
  ja: { title: 'サインイン', email: 'メールアドレス', submit: 'サインイン', ... },
}

const [lang, setLang] = useState('en')
const m = MSGS[lang]

// Language switcher:
[['en', 'ja'].map(l => (
  <button key={l} onClick={() => setLang(l)} aria-pressed={lang === l}>
    {l.toUpperCase()}
  </button>
)
)
)]
```

Full `next-intl` integration with URL-based locale routing (`/ja/login`) is scaffolded in `i18n.js` and `messages/*.json` but requires the `createIntlMiddleware` wrapper to be added to `middleware.js` for complete activation.

16. Security Model

Threat Model

XSS (Cross-Site Scripting)

- React's JSX escapes all string content by default — no `dangerouslySetInnerHTML`
- Log event messages rendered as `{ev.message}` — treated as plain text
- No eval, no innerHTML, no template string injection into DOM

CSRF

- Double-submit cookie pattern (X-CSRF-Token header + readable csrf cookie)

- Checked in both Express middleware (mock) and Next.js Edge Middleware
- SameSite=Strict in development; SameSite=None;Secure required for cross-origin production

Token Theft

- Access token: memory-only, 5-min expiry, lost on tab close
- Refresh token: HttpOnly cookie, rotated on every use, UA-bound

Clickjacking

- X-Frame-Options: DENY header set in next.config.js for all routes
- Share page is the one exception (may legitimately be embedded)

Prototype Pollution

- All external JSON parsed through React state (no Object.assign with untrusted keys)
- idb-keyval only stores values we wrote — no external JSON parsed into plain objects

Supply Chain

- Minimal dependencies (11 runtime dependencies total)
- next@14.2.35 patched for CVE-2025-55184 and CVE-2025-67779

Edge Middleware Security

```
// src/middleware.js - runs on every request before any page handler
export function middleware(request) {
  const { pathname } = request.nextUrl

  // 1. Allow public paths without auth check
  if (PUBLIC.some(p => pathname.startsWith(p))) return NextResponse.next()

  // 2. Auth guard - check refresh cookie exists
  const hasRefresh = request.cookies.has('refresh_token')
  if (!hasRefresh) return NextResponse.redirect(new URL('/login', request.url))

  // 3. CSRF check on mutations
  if (['POST', 'PATCH', 'PUT', 'DELETE'].includes(request.method)) {
    const csrfH = request.headers.get('x-csrf-token')
    const csrfC = request.cookies.get('csrf')?.value
    if (!csrfH || csrfH !== csrfC)
      return NextResponse.json({ error: 'invalid csrf' }, { status: 403 })
  }

  return NextResponse.next()
}
```

The middleware runs at the **Edge** — before Next.js renders anything, before any React code runs. This means unauthenticated users are redirected to `/login` before a single byte of the protected page is sent.

17. Performance Budgets & How They Are Met

Budget Table

Metric	Budget	How met
LCP (/)	$\leq 2.0s$	SSR auth guard renders immediately; tenant cards render after client fetch (~300ms)
INP (live page)	$\leq 200ms$	RingBuffer + rAF batching — no JS task > 5ms at 500 msg/s
CLS	≤ 0.05	Fixed-height skeleton placeholders; no layout shift on data load
Main bundle	≤ 180 KB	RSC eliminates most client JS; App Router code-splits per route
Per-route JS	≤ 90 KB	All routes measured < 10 KB (far under budget)
Memory at 200 msg/s	≤ 250 MB	RingBuffer caps in-memory events; <code>slice(-MAX_EVENTS)</code> on reducer
Time to first event	$\leq 400ms$	WS connects immediately after mount; first rAF flush within ~16ms
Share page Lighthouse	≥ 95	ISR static HTML, no client JS required for render

Route Bundle Sizes (from npm run build)

/	2.41 KB	(SSR guard + CSR shell)
/login	2.56 KB	(form + i18n strings)
/share/[token]	0.15 KB	(ISR, effectively zero JS)
/t/[tenant]/projects	2.31 KB	(project list)
/t/[tenant]/s/[service]	8.20 KB	(WS manager + virtual list — largest route)
/t/[tenant]/dashboards/[id]	5.79 KB	(drag/undo/conflict)
/debug/metrics	2.75 KB	(telemetry viewer)

Every route is well under the 90 KB per-route budget. The live-tail page at 8.2 KB is the largest — it contains the RingBuffer, WebSocketManager, VirtualList, and reducer.

INP Guarantee at 500 msg/s

At 500 msg/s the timing works out as:

- Events arrive: 500/s = 1 event every 2ms
- rAF fires: ~60 times/s = once every 16.7ms
- Events per rAF frame: 500/60 ≈ 8 events
- Batch size limit: 200 (far above actual load — headroom for bursts)
- React update: one `dispatch` call per frame (~1ms)
- VirtualList re-render: ~22 rows × DOM update ≈ 3ms
- Total main thread time per frame: ~4ms out of 16.7ms budget

INP measures the time from user interaction to next paint. Since the main thread is free for ~12ms per frame, user interactions (clicks, typing) are processed immediately.

18. Trade-offs & What Was Not Built

Web Worker for WebSocketManager

What: Move the WS + RingBuffer into a dedicated Web Worker so zero WS processing happens on the main thread.

Why not: Web Workers cannot access React state or DOM directly. Communication requires a `MessageChannel` with serialization overhead. The rAF + ring buffer approach keeps individual main-thread tasks < 5ms — already within the INP budget. The Worker approach would add 3+ days of implementation for marginal gain at current event rates.

The right time to add it: When event rates exceed ~1000 msg/s or when CPU profiling shows WS message parsing consuming > 20% of main thread time.

@tanstack/virtual for the List

What: Replace the custom windowing implementation with `@tanstack/virtual`.

Why not: The custom implementation is ~60 lines and handles fixed-height rows perfectly. `@tanstack/virtual` adds 3KB to the bundle. Its main advantage is dynamic row heights — which would be needed for multi-line stack traces.

The right time to add it: When log events include expandable stack traces that vary in height.

Multi-Tab Token Synchronization (BroadcastChannel)

What: Use the `BroadcastChannel` API to synchronize token rotation across tabs. Currently, if two tabs are open, Tab A's token refresh rotates the refresh token, and Tab B's next refresh fails with 401, logging it out.

Why not: Low-frequency real-world impact (users rarely keep two tabs active). Would add ~30 lines but requires careful coordination of the refresh deduplication logic.

Module Federation for Dashboard Editor

What: Load the dashboard editor as a remote module via Webpack Module Federation, enabling independent deployment by a separate team.

Why not: The editor is tightly coupled to the auth context, reducer, and CSRF token. Federating it would require exposing these as shared singletons — complex and brittle. The bundle size advantage is minimal (~5KB saved). Module Federation makes sense only when different teams independently release different parts.

Service Worker for Offline Share Page

What: Cache the share page in a Service Worker so it works fully offline after one visit.

Why not: Service Worker update flows are notoriously complex to get right without disrupting users. The ISR approach already achieves the performance goal (Lighthouse ≥ 95). This is listed as a stretch goal in the assignment.

19. Deployment Architecture

Development (local)

```
npm run dev
├─ next dev    → http://localhost:3000
└─ node mock/server.js → http://localhost:4000
```

Both run from the same `package.json` via concurrently.

Production

Next.js deploys to **Vercel** (serverless, edge CDN). The mock backend deploys to **Railway** (persistent Node.js process, WebSocket support).

```
User's Browser
|
├─ HTTPS → Vercel CDN → Next.js app (obs-console-xyz.vercel.app)
|                                     └─ SSR pages on Vercel Edge Functions
|                                     └─ ISR pages served from CDN cache
|
└─ WSS/HTTPS → Railway → mock/server.js (obs-mock.up.railway.app)
                                └─ REST API endpoints
                                └─ WebSocket event streams
                                └─ SSE alert streams
```

Key Production Configuration Changes

1. **CORS:** Add Vercel URL to allowed origins in `mock/server.js`
 2. **Cookies:** Change `secure: false` → `secure: true`, `SameSite: 'Strict'` → `SameSite: 'None'`
(required for cross-origin cookies between `vercel.app` and `railway.app`)
 3. **JWT_SECRET:** Use a cryptographically random value (not the dev default)
 4. **Environment variables:** Set `NEXT_PUBLIC_API_BASE` and `NEXT_PUBLIC_WS_BASE` to the Railway domain in Vercel's project settings
-

20. How to Rebuild from Scratch

If you wanted to rebuild this project from scratch with full understanding, here is the order of implementation and the key decisions at each step.

Step 1: Project Scaffold

```
npx create-next-app@14 obs-console --app --no-typescript --no-tailwind
cd obs-console
npm install express ws uuid cors cookie-parser web-vitals idb-keyval next-intl
```

Set up `jsconfig.json` with path aliases first — it will save significant pain with import paths in deeply nested App Router folders.

Step 2: Mock Backend

Build `mock/server.js` first. You need working auth endpoints before you can build any authenticated UI. Order within the mock:

1. JWT sign/verify functions (pure crypto, no React)
2. `POST /api/auth/login` and `POST /api/auth/refresh` with cookie setting
3. `requireAuth` middleware
4. Static data (TENANTS, PROJECTS, SERVICES)
5. REST endpoints (tenants, projects, services)
6. WebSocket upgrade handler + `makeBurstGen`
7. `POST /api/telemetry/ingest` (needed for telemetry later)

Test with curl before touching the frontend:

```
curl -X POST http://localhost:4000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@acme.com","password":"password"}'
```

Step 3: Auth Layer

Build `src/store/auth.js` — the `AuthProvider` with `useReducer`. The key insight: **the access token must never be stored anywhere persistent**. Test that silent refresh works on page reload before moving on.

Build `src/middleware.js` — the Edge Middleware auth guard. Test that navigating to `/` without a cookie redirects to `/login`.

Step 4: Login Page

Simple form with `useAuth().login()`. Verify that after login, the refresh cookie is set and the access token is in context.

Step 5: Landing Page

Build `page.js` (SSR cookie check) + `LandingClient.js` (client-side tenant fetch). This establishes the critical pattern: **server checks auth via cookie; client fetches data via Bearer token**. Every subsequent page follows this same pattern.

Step 6: RingBuffer

Write the 41-line `RingBuffer` class. Test it in isolation: push 6000 events to a 5000-cap ring, verify 1000 are dropped, verify drain returns events in FIFO order.

Step 7: WebSocketManager

Build `wsManager.js`. Test that it connects to the mock WS endpoint, pushes events into the ring, and the `rAF` flush calls `onBatch` with batches of ≤ 200 events. Test reconnect by stopping and restarting the mock server.

Step 8: Live Tail Page

Build `LiveTailShell.js`. Start with the reducer (most complex logic), then the `VirtualList` (most visually complex), then connect them via the `rAF` flush callback. Add ARIA attributes once the basic functionality works.

The most common mistake here: putting `useEffect` for the WebSocket connection inside the component but declaring `useReducer` after it. React's Rules of Hooks require all hooks to be called in the same order every render — declare all hooks at the very top of the component before any conditional logic or side effects.

Step 9: Projects Page + Intercepting Routes

The parallel/intercepting route setup must follow the exact file convention:

```
@modal/default.js           ← required, even if it just returns null
@modal/(..)projects/[id]/page.js ← the intercepting route
layout.js                   ← must accept { children, modal } props
```

Test by clicking a project card (should show modal) vs hard-refreshing `/t/acme/projects/p1` (should show full page).

Step 10: Dashboard Editor

Build the `historyReducer` first and test it with unit assertions:

- SET adds to past, clears future
- UNDO pops past into present, pushes present to future
- REDO pops future into present, pushes present to past
- `past.length` never exceeds `MAX_HISTORY`

Then build the UI: widgets grid, drag handlers, save/conflict flow, offline queue.

Step 11: Share Page + ISR

Simple async Server Component with `revalidate = false`. The key is using `next: { tags: ['share'] }` on the `fetch` so it can be invalidated later.

Step 12: Telemetry

Build `telemetry.js` last because it depends on the app being mostly complete (you need real pages to measure). `initTelemetry()` is called from `TelemetryInit.js` which is rendered in the root layout. Every page automatically benefits.

Build `/debug/metrics` last — it's the viewer for data collected by the telemetry layer.

Step 13: ADRs and Documentation

Write ADRs after the code is complete — they document decisions that were actually made, not hypothetical ones. Each ADR should answer: what was the context, what was decided, what alternatives were considered and rejected, what are the consequences.

Key Insight Summary

The entire project architecture flows from three constraints:

1. **500 msg/s cannot call `setState` directly** → `RingBuffer` + `rAF` batching + `VirtualList`
2. **Access token is memory-only** → every authenticated data fetch must be a Client Component; Server Components can only verify session existence via cookies
3. **Public share page must score ≥95 Lighthouse** → ISR static generation with no client JavaScript required for the initial render

Every other decision — the rendering strategy per route, the state management approach, the offline queue, the telemetry sampling — follows logically from these three constraints. Understanding them deeply means you understand why every file in the project is structured the way it is.